1. Introduction:

Cloudvoyance is a technology company that provides cloud infrastructure, automation, and scalable SaaS solutions for modern applications. The company focuses on cloud-native systems, automation pipelines, and secure multi-cloud environments. Their work demonstrates how SaaS platforms can efficiently manage large-scale systems using reliable cloud architecture.

Inspired by such cloud-based platforms, this project designs a SaaS-based Restaurant Credit Wallet System using relational database concepts. The system allows multiple restaurants to operate on a shared cloud platform while maintaining secure and organized data management.

In this system, customers can maintain a digital wallet, load money, and pay for their food by scanning a QR code at their table. The database ensures that all wallet transactions and order payments are processed safely using ACID properties to maintain consistency and prevent errors.

This project applies key database concepts such as entity-relationship modeling, relational schema design, primary and foreign keys, composite keys, stored procedures, triggers, and transaction management. The main objective is to design a secure, scalable, and efficient backend database system for real-time QR-based restaurant payments in a SaaS environment.

2. Extended Entity Relationship Model

An Extended Entity-Relationship Diagram (EERD) is an advanced version of the traditional Entity-Relationship Diagram (ERD) used in database design. EERDs include additional concepts and features to model more complex relationships and constraints in a database schema. Here are some key elements and notations commonly used in an Extended Entity-Relationship Diagram:

1. **Entities**: Entities represent real-world objects or concepts. They are typically depicted as rectangles with the entity name inside.
2. **Attributes**: Attributes describe the properties or characteristics of entities. They are shown as ovals connected to their respective entities by lines. Each attribute has a name and a data type.

3. **Key Attributes**: Key attributes uniquely identify each entity within its set. They are underlined or marked with a key symbol.
4. **Derived Attributes**: Derived attributes are attributes whose values can be calculated from other attributes in the database. They are represented with a dashed oval.
5. **Relationships**: Relationships describe how entities are related to each other. They are represented as diamond shapes connecting the related entities. The cardinality and participation constraints of relationships can be indicated near the diamonds.
6. **Cardinality Constraints**: These constraints specify the number of instances of one entity that can be related to the other entity. Common notations are "1" (one) and "M" (many).
7. **Participation Constraints**: These constraints specify whether the participation of entities in a relationship is mandatory (total) or optional (partial).
8. **Roles**: In some cases, the same entity can participate in multiple roles within a single relationship. Roles help clarify the purpose of an entity within a specific relationship.
9. **Multivalued Attributes**: Multivalued attributes are attributes that can have multiple values for a single entity. They are represented with a double oval.
10. **Weak Entities**: Weak entities are entities that do not have a primary key attribute on their own. They rely on a relationship with a strong entity (called the identifying or owning entity) for identification.
11. **Specialization and Generalization**: EERDs allow for the modeling of hierarchical relationships between entities through specialization and generalization. An entity can be specialized into subtypes, and generalization represents a higher-level entity that encompasses multiple subtypes.
12. **Union Types**: Union types are used to represent an entity that can be one of several entity types. It is typically depicted as a circle with labeled arcs pointing to the possible entity types.
13. **Aggregation**: Aggregation represents a higher-level entity composed of related entities. It is depicted as a diamond shape with a line connecting it to the aggregated entities.

14. **Constraints**: Constraints can be added to specify additional rules and requirements for the database schema, such as uniqueness constraints, referential integrity constraints, or check constraints.

Extended Entity-Relationship Diagrams are particularly useful for designing complex databases with intricate relationships and constraints. They provide a more comprehensive and structured way to model data compared to traditional ERDs, making them valuable tools in database design and development.

3. EER diagrams:

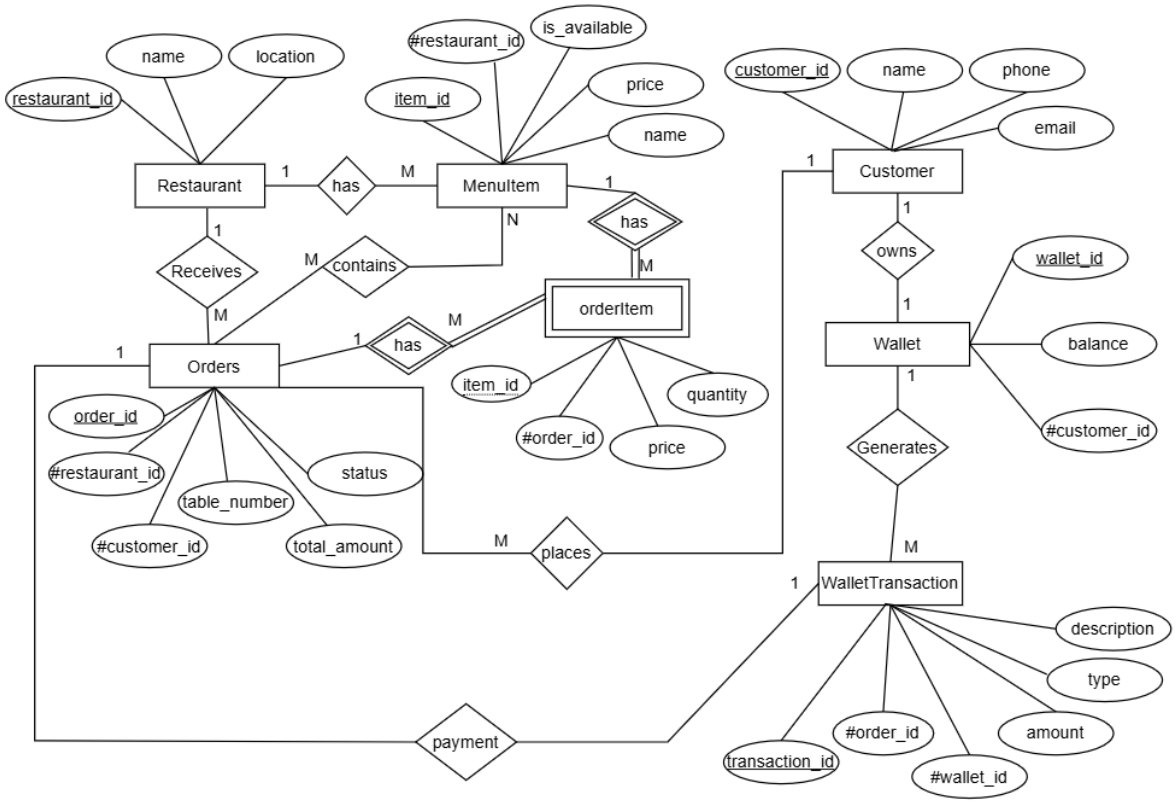


Figure 1: EER diagram of Restaurant

Mapping the EER diagram to Relational table

The relational model represents how data is stored in Relational Databases. A relational database consists of a collection of tables, each of which is assigned a unique name. The relational model can represent a table with columns and rows. Each row is known as a tuple and each table of the column has a name or attribute. To establish relationships between tables, foreign keys (FK) are used, which reference the primary key of another table. In some notations, foreign keys are indicated with a # before the attribute name. For example, in a Wallet table, wallet_id would be the primary key, and #customer_id would be a foreign key referencing the Customer(customer_id) table. This shows that each wallet belongs to a specific customer, ensuring the relationship between the two tables.

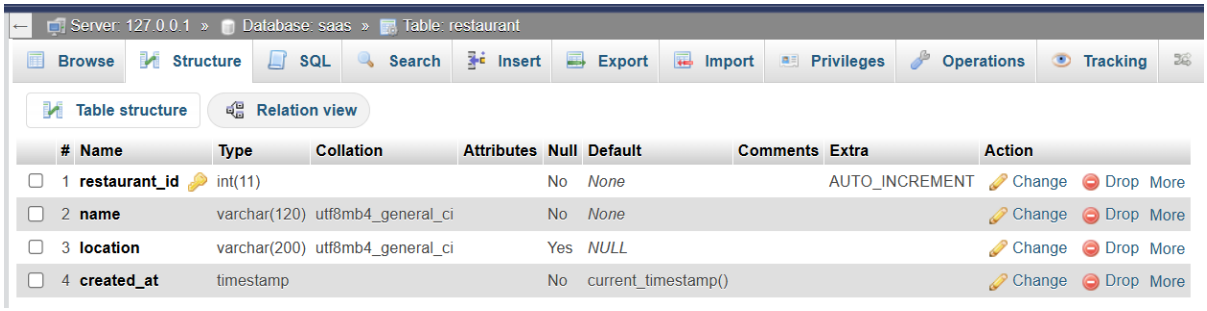
Properties of Relation:

1. The name of the relation is distinct from all other relations.
2. Each relation cell contains exactly one atomic (single) value
3. Each attribute contains a distinct name
4. The attribute domain has no significance
5. Tuple has no duplicate value
6. The order of tuples can have a different sequence

A database is an organized collection of structured information, or data, typically stored electronically in a computer system. A database is usually controlled by a database management system (DBMS). Together, the data and the DBMS, along with the applications that are associated with them, are referred to as a database system, often shortened to just a database.

Data within the most common types of databases in operation today is typically modeled in rows and columns in a series of tables to make processing and data querying efficient. The data can then be easily accessed, managed, modified, updated, controlled, and organized. Most databases use structured query language (SQL) for writing and querying data.

Restaurant



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	restaurant_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	name	varchar(120)	utf8mb4_general_ci		No	None			Change Drop More
3	location	varchar(200)	utf8mb4_general_ci		Yes	NULL			Change Drop More
4	created_at	timestamp			No	current_timestamp()			Change Drop More

Customer

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	customer_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	name	varchar(120)	utf8mb4_general_ci		No	None			Change Drop More
3	phone	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
4	email	varchar(150)	utf8mb4_general_ci		Yes	NULL			Change Drop More
5	created_at	timestamp			No	current_timestamp()			Change Drop More

Wallet

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	wallet_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	customer_id	int(11)			No	None			Change Drop More
3	balance	decimal(12,2)			Yes	0.00			Change Drop More
4	updated_at	timestamp			No	current_timestamp()		ON UPDATE CURRENT_TIMESTAMP()	Change Drop More

Menu Item

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	item_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	restaurant_id	int(11)			No	None			Change Drop More
3	name	varchar(120)	utf8mb4_general_ci		No	None			Change Drop More
4	price	decimal(10,2)			No	None			Change Drop More
5	is_available	tinyint(1)			Yes	1			Change Drop More

Orders

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	order_id	bigint(20)			No	None		AUTO_INCREMENT	Change Drop More
2	customer_id	int(11)			No	None			Change Drop More
3	restaurant_id	int(11)			No	None			Change Drop More
4	table_number	int(11)			Yes	NULL			Change Drop More
5	total_amount	decimal(12,2)			Yes	0.00			Change Drop More
6	status	enum('PENDING', 'PAID', 'CANCELLED')	utf8mb4_general_ci		Yes	PENDING			Change Drop More
7	created_at	timestamp			No	current_timestamp()			Change Drop More

Order Item

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	order_id	bigint(20)			No	None			Change Drop More
2	item_id	int(11)			No	None			Change Drop More
3	quantity	int(11)			No	None			Change Drop More
4	price	decimal(10,2)			No	None			Change Drop More

Wallet Transaction

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	transaction_id	bigint(20)			No	None		AUTO_INCREMENT	Change Drop More
2	wallet_id	int(11)			No	None			Change Drop More
3	order_id	bigint(20)			Yes	NULL			Change Drop More
4	amount	decimal(12,2)			No	None			Change Drop More
5	type	enum('CREDIT', 'DEBIT')	utf8mb4_general_ci		No	None			Change Drop More
6	description	varchar(200)	utf8mb4_general_ci		Yes	NULL			Change Drop More
7	created_at	timestamp			No	current_timestamp()			Change Drop More

Relation Model of SAAS Application:

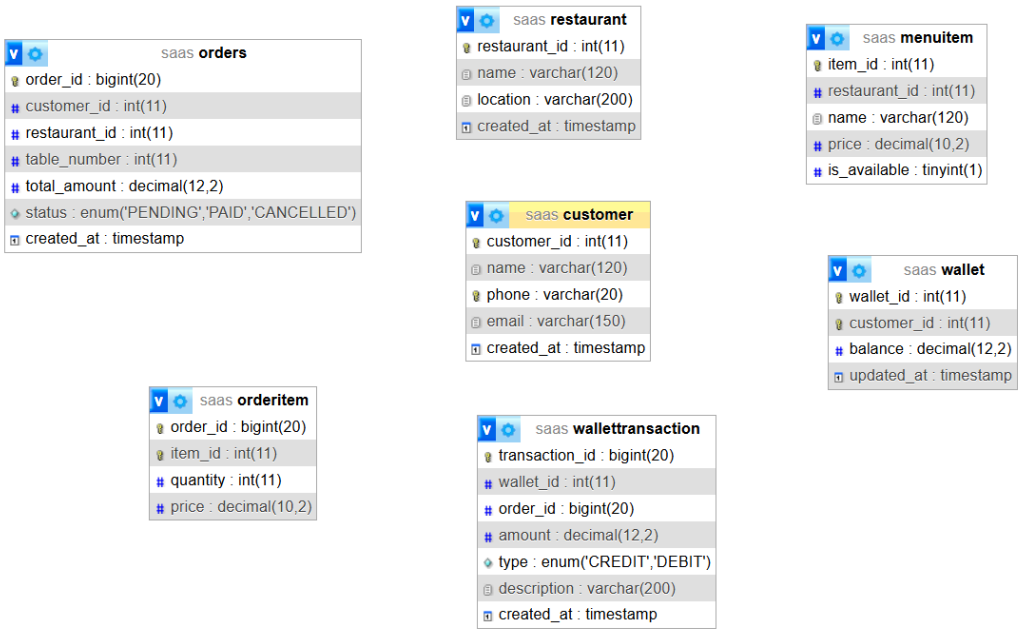
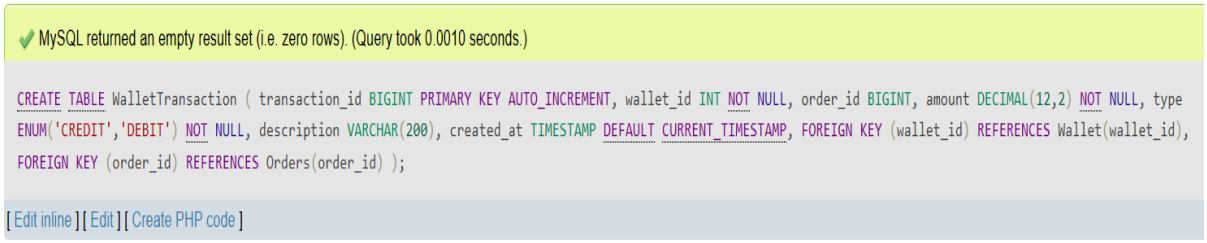


Figure 2: Designer Relationship of Saas application

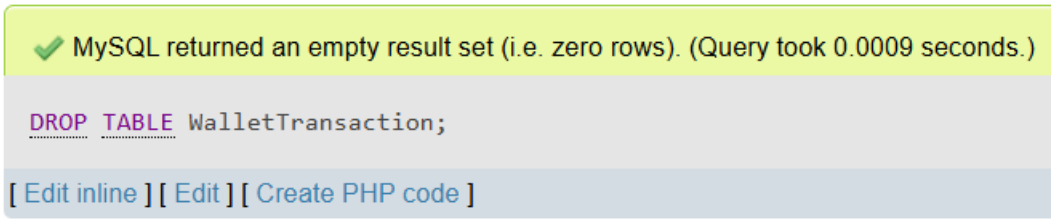
SQL commands in Relational Databases with stored procedure and Triggers:

Create table wallet transition:

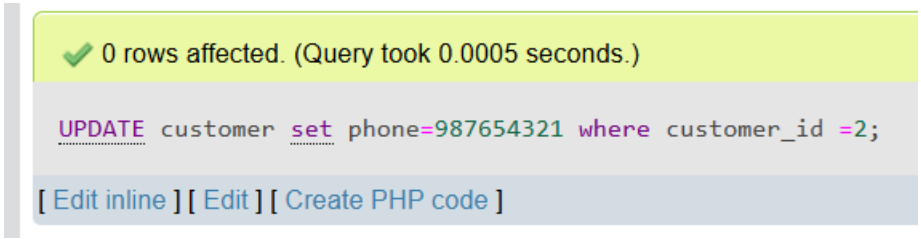


Insert multiple data into database:

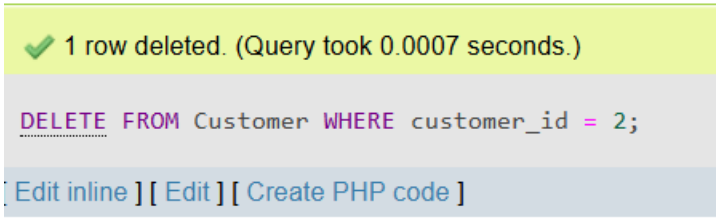
Drop the table:



To update data:



To delete records:



To alter the table:

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0011 seconds.)

```
ALTER TABLE Customer ADD COLUMN address VARCHAR(200);
```

[Edit inline] [Edit] [Create PHP code]

Join:

Showing rows 0 - 19 (20 total, Query took 0.0003 seconds.)

```
SELECT c.name AS customer_name, r.name AS restaurant_name FROM Customer c CROSS JOIN Restaurant r;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

customer_name	restaurant_name
philip magar	The Spice House
sajan magar	The Spice House
sujan Thapa	The Spice House
sugam pokharel	The Spice House
philip magar	Green Garden Cafe
sajan magar	Green Garden Cafe
sujan Thapa	Green Garden Cafe
sugam pokharel	Green Garden Cafe
philip magar	Ocean Breeze Restaurant
sajan magar	Ocean Breeze Restaurant
sujan Thapa	Ocean Breeze Restaurant
sugam pokharel	Ocean Breeze Restaurant

Inner join:

Showing rows 0 - 4 (5 total, Query took 0.0003 seconds.)

```
SELECT o.order_id, c.name AS customer_name, o.total_amount, o.status FROM Orders o INNER JOIN Customer c ON o.customer_id = c.customer_id;
```

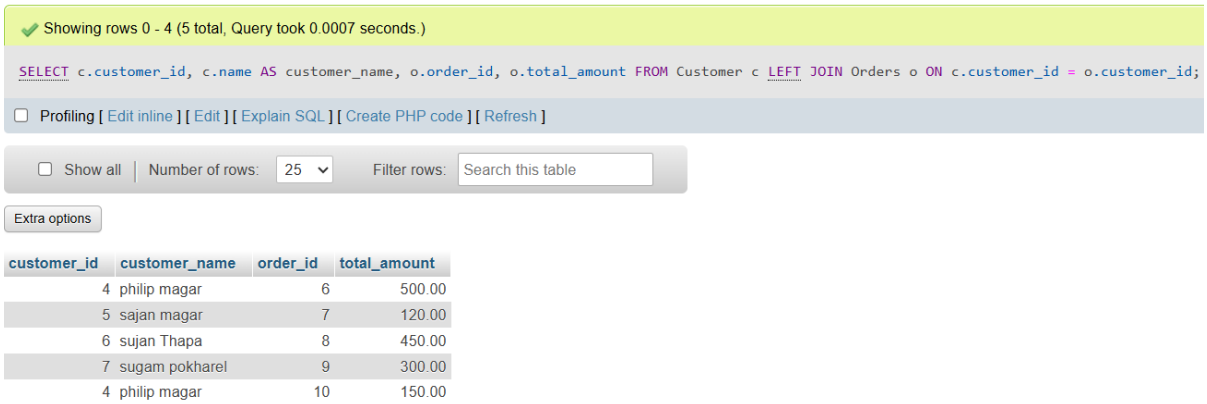
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

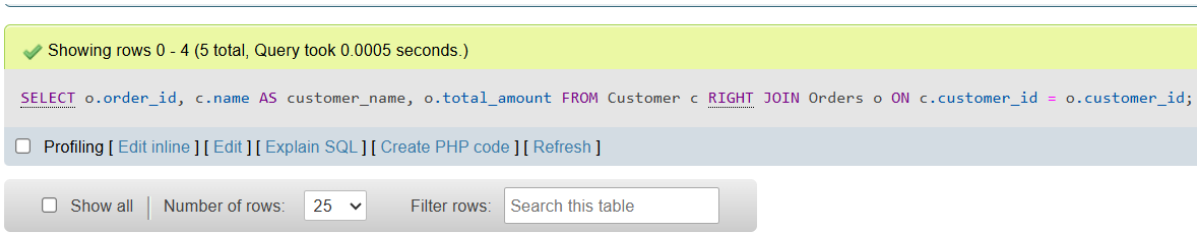
order_id	customer_name	total_amount	status
6	philip magar	500.00	PENDING
7	sajan magar	120.00	PAID
8	sujan Thapa	450.00	PENDING
9	sugam pokharel	300.00	CANCELLED
10	philip magar	150.00	PAID

Left join:



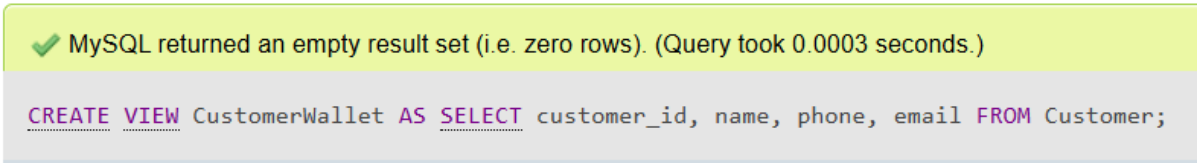
customer_id	customer_name	order_id	total_amount
4	philip magar	6	500.00
5	sajan magar	7	120.00
6	sujan Thapa	8	450.00
7	sugam pokharel	9	300.00
4	philip magar	10	150.00

Right join

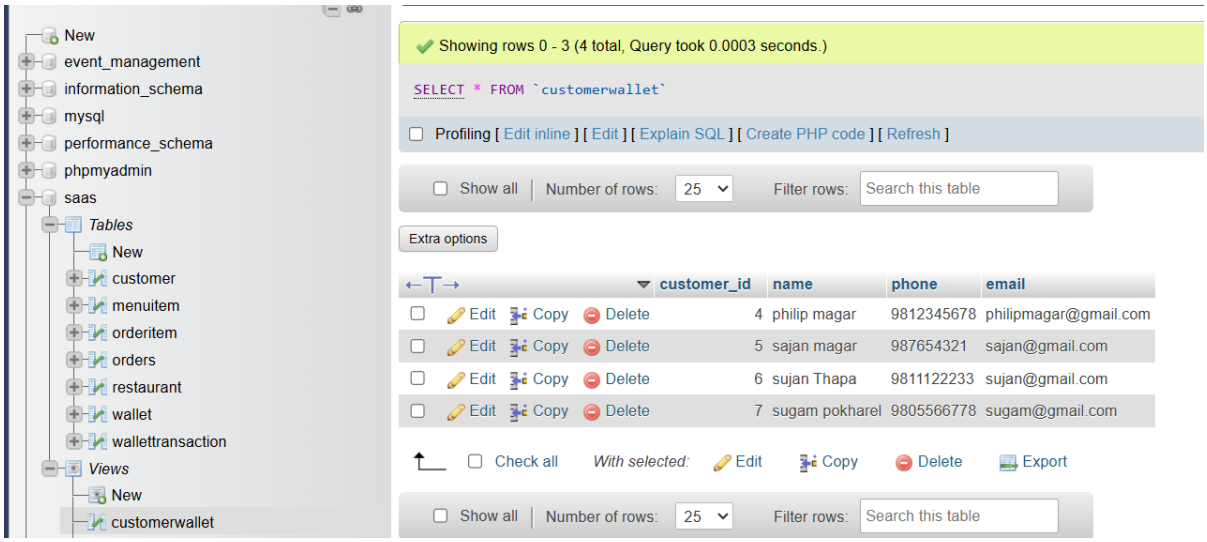


order_id	customer_name	total_amount
6	philip magar	500.00
7	sajan magar	120.00
8	sujan Thapa	450.00
9	sugam pokharel	300.00
10	philip magar	150.00

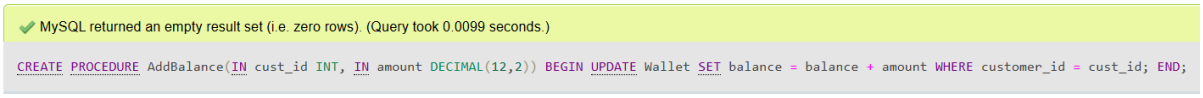
View:

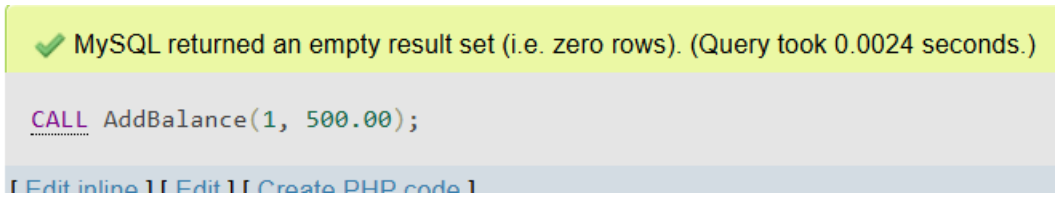


Selecting data from view:



Creating procedure:





Creating trigger:

```
DELIMITER $$

CREATE TRIGGER after_customer_insert

AFTER INSERT ON Customer

FOR EACH ROW

BEGIN

    INSERT INTO Wallet (customer_id, balance)

    VALUES (NEW.customer_id, 0);

END $$
```

DELIMITER ;

```
CREATE TRIGGER after_customer_insert AFTER INSERT ON Customer FOR EACH ROW BEGIN INSERT INTO Wallet (customer_id, balance) VALUES (NEW.customer_id, 0); END;
```

[Edit inline] [Edit] [Create PHP code]

Inserting data after creating trigger:

✔ 1 row inserted.
Inserted row id: 8 (Query took 0.0003 seconds.)

```
INSERT INTO Customer (name, phone, email) VALUES ('Niraj Lama', '+9779812233445', 'niraj@gmail.com');
```

[Edit inline] [Edit] [Create PHP code]

Selecting data from customer:

✔ Showing rows 0 - 4 (5 total, Query took 0.0002 seconds.)

```
SELECT * FROM Customer;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	customer_id	name	phone	email	created_at
Edit Copy Delete	4	philip magar	9812345678	philipmagar@gmail.com	2026-03-09 20:49:16
Edit Copy Delete	5	sajan magar	987654321	sajan@gmail.com	2026-03-09 20:49:16
Edit Copy Delete	6	sujan Thapa	9811122233	sujan@gmail.com	2026-03-09 20:49:16
Edit Copy Delete	7	sugam pokharel	9805566778	sugam@gmail.com	2026-03-09 20:49:16
Edit Copy Delete	8	Niraj Lama	+9779812233445	niraj@gmail.com	2026-03-09 21:10:48

Subquery with exists:

Showing rows 0 - 4 (5 total, Query took 0.0028 seconds.)

```
SELECT * FROM Customer c WHERE EXISTS ( SELECT 1 FROM Wallet w WHERE w.customer_id = c.customer_id );
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	customer_id	name	phone	email	created_at
Edit Copy Delete	4	philip magar	9812345678	philipmagar@gmail.com	2026-03-09 20:49:16
Edit Copy Delete	5	sajan magar	987654321	sajan@gmail.com	2026-03-09 20:49:16
Edit Copy Delete	6	sujan Thapa	9811122233	sujan@gmail.com	2026-03-09 20:49:16
Edit Copy Delete	7	sugam pokharel	9805566778	sugam@gmail.com	2026-03-09 20:49:16
Edit Copy Delete	8	Niraj Lama	+9779812233445	niraj@gmail.com	2026-03-09 21:10:48

Subquery with Not exists:

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds.)

```
SELECT * FROM Customer c WHERE NOT EXISTS ( SELECT 14 FROM Wallet w WHERE w.customer_id = c.customer_id );
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

customer_id	name	phone	email	created_at	address

Introduction to MongoDB

MongoDB is a popular, open-source NoSQL database that is designed for flexibility, scalability, and ease of development. It belongs to the document-oriented database category and is known for its ability to handle unstructured or semi-structured data. Here's an overview of key concepts and features related to MongoDB:

1. **Document-Oriented Database**: MongoDB stores data in a format called BSON (Binary JSON), which is a binary representation of JSON-like documents. Each document can have a different structure, making MongoDB flexible for accommodating data of varying shapes.
2. **Collections**: MongoDB organizes data into collections, which are similar to tables in relational databases. Collections can contain multiple documents, and documents within a collection do not need to have the same fields. .
3. **Fields**: Fields are key-value pairs within a document. Field names are case-sensitive.
4. **ObjectId**: MongoDB automatically assigns a unique ObjectId to each document as its primary key. This field is typically named _id.
5. **Indexes**: MongoDB supports indexing for efficient querying. Indexes can be created on single fields, compound fields, or arrays to improve query performance.
6. Query **Language**: MongoDB uses a powerful query language for searching and filtering documents. Queries can use a wide range of operators, including equality, comparison, and logical operators.
7. **Replication**: MongoDB supports replica sets, which are clusters of MongoDB servers that maintain the same data for redundancy and high availability. Replica sets provide automatic failover and data redundancy.
8. **Sharding**: MongoDB can horizontally scale by using sharding. Sharding divides data across multiple servers based on a shard key, allowing for distribution of data and queries.
9. **Transactions**: Starting with MongoDB 4.0, MongoDB supports multi-document transactions, making it suitable for applications requiring ACID (Atomicity, Consistency, Isolation, Durability) transactions.

10. **Geospatial Data**: MongoDB has built-in support for geospatial data and geospatial indexing, allowing for location-based queries.
11. **Security**: MongoDB offers various security features, including authentication, authorization, and encryption. Role-based access control (RBAC) can be used to manage user permissions.

Installation of MongoDB

To install MongoDB on a Windows operating system, follow these step-by-step instructions:

Step 1: Download MongoDB

- Visit the official MongoDB website at https://www.mongodb.com/try/download/community.
- Scroll down to the "Community Server" section, where you can download the MongoDB software for free.
- Under the "Windows" subsection, locate and click on the "Download" button to initiate the download of the latest stable version of MongoDB for Windows.

Step 2: Run the Installer

- After the download is complete, locate the downloaded installer file, typically found in the computer's "Downloads" folder.
- Double-click on the installer file to execute it and begin the installation process**.**

Step 3: Choose Setup Type

- The MongoDB Setup Wizard will appear on the screen. Click the "Next" button to proceed.
- Select the "Complete" setup type, which includes both the MongoDB server and associated tools. Click "Next" to continue.

Step 4: Service Configuration

- You will be presented with the option to configure MongoDB to run as a Windows service. It is recommended to leave this option enabled, as it ensures that MongoDB starts automatically when the computer boots up.
- Optionally, customize the data directory, but most users can stick with the default location. Click "Next" to proceed.

Step 5: Install MongoDB Compass (Optional)

- MongoDB Compass is a graphical user interface (GUI) for MongoDB. Decide whether to install it or not. If chosen, click "Next."

Step 6: Ready to Install

- Review the installation settings displayed on the screen. If everything looks correct, click the "Install" button to initiate the installation process.

Step 7: Installation Progress

- The installer will now copy the necessary files and set up MongoDB on the Windows system. This may take a few minutes to complete**.**

Step 8: Completing the MongoDB Setup

- Once the installation is finished, a screen labeled "Completing the MongoDB Setup" will appear. Decide whether to install MongoDB Compass (the GUI) by checking or unchecking the corresponding option. Then, click "Next" to proceed.

Step 9: Installation Complete

- A screen indicating the successful installation will appear. Ensure that the "Run the MongoDB shell" option is checked if you want to open the MongoDB shell immediately. Then, click "Finish" to finalize the installation.

Step 10: Verify Installation

- To verify that MongoDB has been successfully installed, open either the Windows Command Prompt (CMD) or PowerShell.
- Type "mongo" and press Enter. If MongoDB is installed correctly, this action will open the MongoDB shell, confirming the successful installation.

Setup command for run the queries

To run queries in MongoDB, one typically utilizes the MongoDB shell, a command-line interface for interacting with MongoDB. The following are the basic steps to execute queries using the MongoDB shell:

Step 1: Opening the MongoDB Shell

- Open the command prompt or terminal window.

Step 2: Connecting to the MongoDB Server

- Utilize the "mongo" command followed by the hostname and port of the MongoDB server to establish a connection. MongoDB typically runs on the localhost at port 27017 by default. If MongoDB is hosted on a different server or port, replace the placeholders accordingly.
  mongo --host hostname --port port
  For instance, to connect to the default MongoDB instance running locally: mongo

Step 3: Selecting a Database

- After successfully connecting to the MongoDB server, select a specific database using the "use" command. use your_database_name
- Replace "your_database_name" with the name of the database to be used. MongoDB will create the database if it does not already exist.

Step 4: Executing Queries

- Once a database has been selected, MongoDB queries can be executed. Below are some common query examples:

1. Finding documents in a collection: db.your_collection_name.find({ key: value })
2. Inserting a document into a collection: db.your_collection_name.insert({ key: value })
3. Updating documents in a collection: db.your_collection_name.update({key: value},{ $set:{ new_key: new_value }})
4. Deleting documents from a collection: db.your_collection_name.remove({ key: value }) • Replace "your_collection_name," "key," "value," "new_key," and "new_value" with the actual collection and field names, as well as the specific values intended for querying or modification.

Step 5: Exiting the MongoDB Shell

- When finished working with the MongoDB shell, exit by typing "exit" or pressing "Ctrl + C."

Run NOSQL COMMANDS

Create database:

```
> use RestaurantDB;
< switched to db RestaurantDB
RestaurantDB>
```

Insert in user collection and find data:

```
RestaurantDB> db.users.insertMany([
                { _id: 1, name: "philip", phone: "987654321", email: "philip@example.com" },
                { _id: 2, name: "sajan", phone: "9876543210", email: "bob@example.com" }
              ]);
```

```
> db.users.find();
< {
    _id: 1,
    name: 'philip',
    phone: '987654321',
    email: 'philip@example.com'
  }
  {
    _id: 2,
    name: 'sajan',
    phone: '9876543210',
    email: 'sajan@example.com'
  }
```

Insert in collection admin and find data:

```
> db.admin.insertMany([
    { _id: 1, name: "Admin1", email: "admin1@example.com", role: "manager" }
  ]);
< {
    acknowledged: true,
    insertedIds: {
      '0': 1
    }
  }
```

```
> db.admin.find();
< {
    _id: 1,
    name: 'Admin1',
    email: 'admin1@example.com',
    role: 'manager'
  }
RestaurantDB>
```

Insert in collection products and find the data:

```
> db.products.insertMany([
    { _id: 103, name: "momo", price: 100, description: "momo" },
    { _id: 104, name: "chowmein", price: 100, description: "chowmein" }
  ]);
< {
    acknowledged: true,
    insertedIds: {
      '0': 103,
      '1': 104
    }
  }
```

```
  {
    _id: 103,
    name: 'momo',
    price: 100,
    description: 'momo'
  }
  {
    _id: 104,
    name: 'chowmein',
    price: 100,
    description: 'chowmein'
  }
RestaurantDB>
```

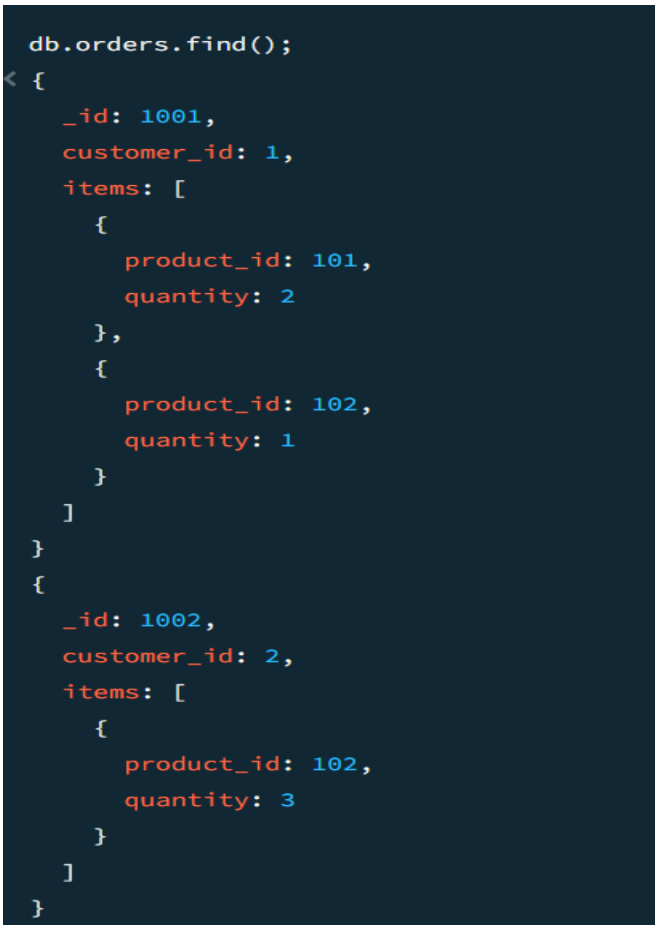
Insert in collection category and find the data:

```
> db.category.insertMany([
    { _id: 1, name: "Fast Food", description: "momo, chowmein, sasuages" },
    { _id: 2, name: "momo", description: "Veg, Buff, Chicken" }
  ]);
< {
    acknowledged: true,
    insertedIds: {
      '0': 1,
      '1': 2
    }
  }
```

```
> db.category.find();
< {
    _id: 1,
    name: 'Fast Food',
    description: 'momo, chowmein, sasuages'
  }
  {
    _id: 2,
    name: 'momo',
    description: 'Veg, Buff, Chicken'
  }
```

Insert in collection orders and find the data:

```
> db.orders.insertMany([
   {
     _id: 1001,
     customer_id: 1,
     items: [
       { product_id: 101, quantity: 2 },
       { product_id: 102, quantity: 1 }
     ]
   },
   {
     _id: 1002,
     customer_id: 2,
     items: [
       { product_id: 102, quantity: 3 }
     ]
   }
 ]);
```



Q. Find the User name, email, contact number, product name and product price and product description for every order:

```
db.orders.aggregate([
  {
   $lookup: {
     from: "users",
     localField: "customer_id",
     foreignField: "_id",
     as: "customer"
   }
```

```
},
{ $unwind: "$customer" },
{
 $lookup: {
   from: "products",
   localField: "items.product_id",
   foreignField: "_id",
   as: "products"
 }
},
{
 $project: {
   _id: 0,
   customer_name: "$customer.name",
   customer_email: "$customer.email",
   customer_phone: "$customer.phone",
   products: {
     $map: {
       input: "$products",
       as: "p",
       in: { name: "$$p.name", price: "$$p.price", description: "$$p.description" }
     }
   }
```

```
  }
 }
]);
```

Output:

```
< {
    customer_name: 'philip',
    customer_email: 'philip@example.com',
    customer_phone: '987654321',
    products: [
      {
        name: 'momo',
        price: 100,
        description: 'Delicious burger'
      },
      {
        name: 'chowmein',
        price: 100,
        description: 'Cheesy pizza'
      }
    ]
  }
  {
    customer_name: 'sajan',
    customer_email: 'sajan@example.com',
    customer_phone: '9876543210',
    products: [
      {
        name: 'chowmein',
        price: 100,
        description: 'Cheesy pizza'
      }
    ]
```

Conclusion:

In this Advanced Database practical, we learned about following:

- Learned how to designed Extended Entity Relationship (EER) Diagram.
- Get Knowledge of how EER diagram is converted to relational table. • Had idea of how EER is implemented in SQL
- Learned about various SQL queries for SQL Data Definition language (create, alter, drop) and Data Manipulation language (select, insert, alter, aggregate, join etc.) queries.
- Get idea of Trigger and Active Procedure.
- Had knowledge of JOIN, view and procedure.
- Learned about MongoDB installation and create of database and collections.
- Get idea of how data are inserted in collections and find the data.